



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 10

Explorando nuestros datos con Pandas

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Resultados de aprendizaje

- Describir el **flujo de datos** habitual en Ciencia de Datos para manipular y analizar conjuntos de datos (*datasets*)
- Explicar los conceptos de **Series** y **DataFrames**, así como sus principales características
- Identificar las **propiedades** y **métodos** fundamentales de Series y DataFrames
- **Operar** con Series y DataFrames de forma correcta
- Ser capaz de **combinar** y **agrupar DataFrames**
- Ser capaz de **manipular** estructuras de datos complejas para **analizar** sus principales características

Índice



- 1. Introducción**
- 2. Series**
- 3. DataFrames**
- 4. Combinar DataFrames**
- 5. Agrupar DataFrames**
- 6. Preprocesado de datos**

Índice



1. Introducción

2. Series

3. DataFrames

4. Combinar DataFrames

5. Agrupar DataFrames

6. Preprocesado de datos

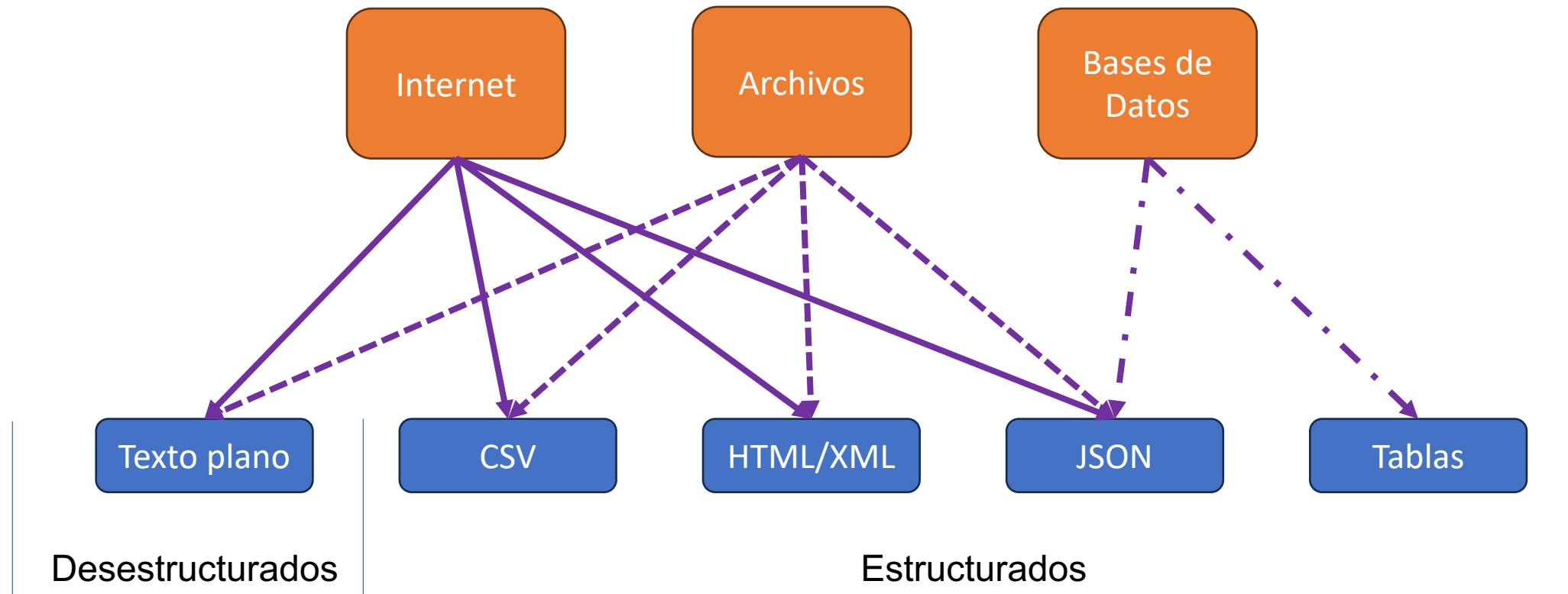
Introducción

El flujo (*pipeline*) de los datos

- Trabajar con Ciencia de Datos y con Inteligencia Artificial implica manipular datos, **muchos datos**
- En el mundo real los datos nunca están en el formato deseable
- Los datos vienen habitualmente de diversas **fuentes**, en diferentes **tipos** y, generalmente, con **errores**
- Es necesario **manipularlos, ordenarlos, arreglar** los errores y **visualizarlos** para evitar problemas posteriores
- Los datos pueden estar en formatos **estructurados o no estructurados**

Introducción

El flujo de los datos



Introducción

Conociendo Pandas

- Las **tablas** son una forma de representar datos muy sencilla para las personas
- Pandas es un módulo de Python que implementa una forma de manipular datos **tabulares**
- Está basado en **NumPy** pero centrado en análisis de datos más complejos
- Es capaz de manejar diferentes **tipos** de datos
- Es un pilar **fundamental** de la Ciencia de Datos en Python

Introducción

Pandas para el análisis de datos

Biblioteca (*librería*) esencial para el análisis de datos

- Representación efectiva y resumen de datos con base **estadística**
- Alineación y manipulación de datos para **combinar** y **cruzar** diferentes fuentes
- Procesamiento **flexible**: recopilación, transformación y aplicación de funciones sobre conjuntos completos de datos
- Mientras que NumPy se centra en cálculos numéricos sobre *arrays* homogéneos, Pandas está diseñado para **datos tabulares** (tipo hoja de cálculo), facilitando análisis estadístico y manejo de series temporales

Introducción

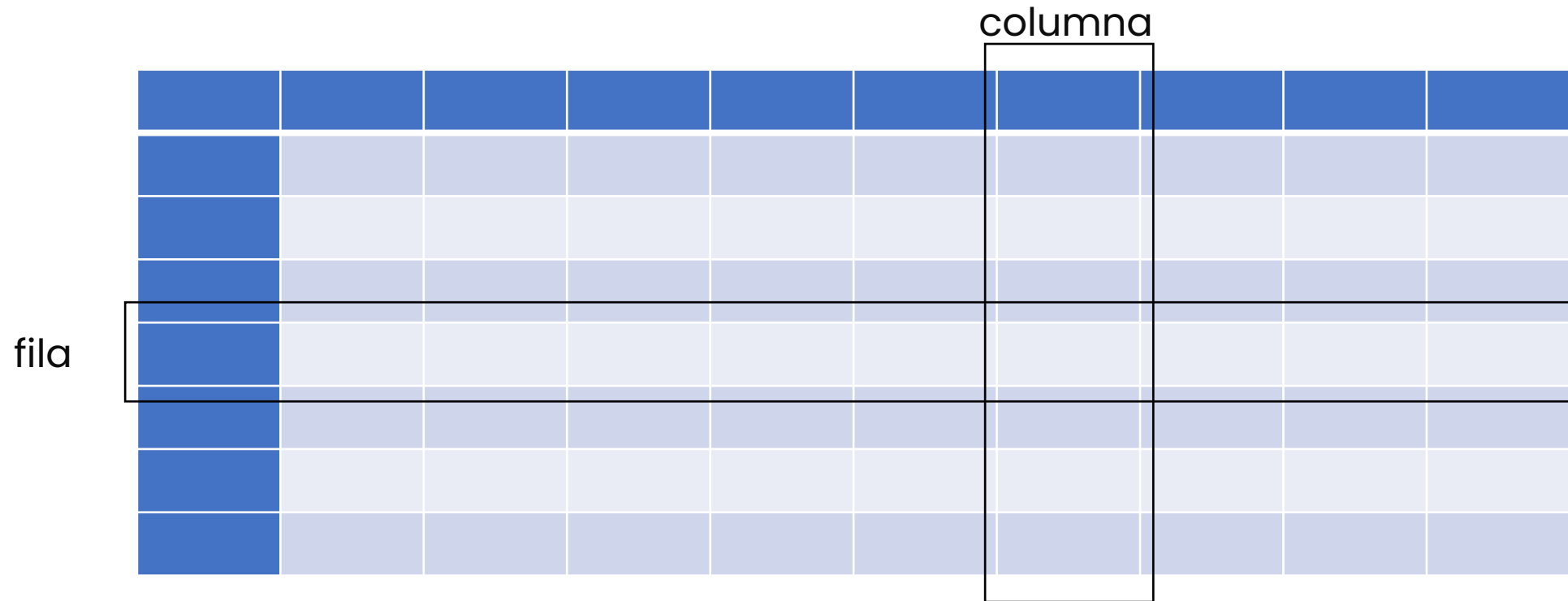
Las estructuras de datos de Pandas

Series		Series		Series		Series
Sex		Bloodtype		Height		Weight
Male		3		23		68.2
Female		2		22		53
Male		4		24		80.1
Male	+	3	+	23	+	85.7
Female		1		20		49.5
Female		2		21		52
Female		1		22		45.3
DataFrame						

Las estructuras de datos básicas de pandas son **Series** (1D) y **DataFrame** (2D).
Un DataFrame es un contenedor de Series, y una Series es un contenedor de escalares (0D).

Introducción

Las estructuras de datos de Pandas



Cada columna de un DataFrame es una Series. Las filas se indexan por el índice de las series.

Índice



1. Introducción

2. Series

3. DataFrames

4. Combinar DataFrames

5. Agrupar DataFrames

6. Preprocesado de datos

Series

Introducción a las Series

Una Series es un tipo de **array unidimensional** con propiedades **extendidas**:



Características básicas:

- Todos los elementos de una Series deben ser del mismo tipo de dato
- Se pueden crear Series a partir de secuencias como listas, tuplas o arrays
- La palabra “series” se usa tanto en singular como en plural (del latín *serere*, que significa “unir/conectar”)



Índices por defecto:

- El índice inicial de una Series es un entero que comienza en 0
- La primera etiqueta es 0, la segunda 1, y así sucesivamente

Series

Introducción a las Series



Atributos principales:

- `.values` → devuelve todos los valores de la Series
- `.index` → devuelve el índice de la Series
- `.index.values` → devuelve un array con los valores del índice
- `.size` → devuelve la cantidad de valores que contiene

Series

Creando una Series

```
[1]: import pandas as pd
```

```
[2]: pd.Series(["Male", "Female", "Male", "Male", "Female", "Female", "Female"])
```

```
[2]: 0      Male  
     1     Female  
     2      Male  
     3      Male  
     4     Female  
     5     Female  
     6     Female  
     dtype: object
```

Se puede crear una Series a partir de una lista de Python

Series

Creando una Series

```
[4]: import pandas as pd  
import numpy as np
```

```
[5]: pd.Series(np.array(["Male", "Female", "Male", "Male", "Female", "Female", "Female"]))
```

```
[5]: 0    Male  
1    Female  
2    Male  
3    Male  
4    Female  
5    Female  
6    Female  
dtype: object
```

También se puede crear una Series a partir de un NumPy array

Series

Creando una Series

```
[6]: import pandas as pd
```

```
[8]: series = pd.Series(tuple(["Male", "Female", "Male", "Male", "Female", "Female", "Female"]))
```

```
[9]: type(series)
```

```
[9]: pandas.core.series.Series
```

Cualquier “iterable” sirve para crear una Series (tuplas, sets, listas, etc.)

Series

Creando una Series

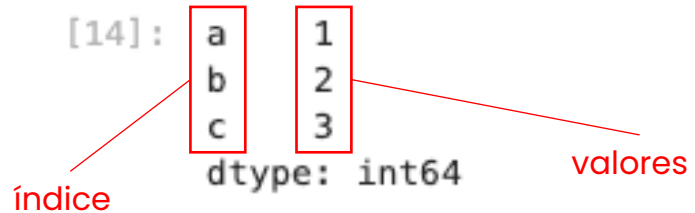
```
[13]: dict_data = {"a": 1, "b": 2, "c": 3}
```

```
[14]: pd.Series(dict_data)
```

```
[14]: a    1  
      b    2  
      c    3  
      dtype: int64
```

índice

valores

A diagram illustrating the creation of a pandas Series from a dictionary. The dictionary is {"a": 1, "b": 2, "c": 3}. The resulting Series is displayed with indices 'a', 'b', 'c' and values 1, 2, 3. Red boxes highlight the indices and values columns. A red line points from the word 'índice' to the index column, and another red line points from the word 'valores' to the values column. The data type 'dtype: int64' is shown at the bottom.

Si creamos una Series a partir de un **diccionario**:

- Las claves (keys) se usan como índice de la Series
- Los valores (values) se usan como valores de la Series

Series

Creando una Series

```
[13]: dict_data = {"a": 1, "b": 2, "c": 3}
```

```
[15]: ser1 = pd.Series(dict_data)
```

```
[16]: ser1.index
```

```
[16]: Index(['a', 'b', 'c'], dtype='object')
```

```
[17]: ser1.values
```

```
[17]: array([1, 2, 3])
```

Podemos extraer el índice y los valores por separado:

La clase **Index** almacena índices que no sean números secuenciales

Los valores se almacenan en un NumPy array

Series

Creando una Series

```
[18]: ser1 = pd.Series(["2019-15-04", 3.14, "ABC", 100, True])
```

```
[21]: ser1
```

```
[21]: 0    2019-15-04  
      1         3.14  
      2         ABC  
      3         100  
      4         True  
      dtype: object
```

```
[22]: ser1.index
```

```
[22]: RangeIndex(start=0, stop=5, step=1)
```

Si los datos no son homogéneos pandas los tratará de igualar (en el ejemplo a strings)

Un índice compuesto por un rango de enteros será de tipo **RangeIndex**.

Series

Creando una Series

```
[51]: valores = [220, 215, 93, 64, 64]
```

```
[52]: pd.Series(data=valores, index=['Brown', 'Blue', "Gray", 'Hazel', 'Green'])
```

```
[52]: Brown      220  
      Blue      215  
      Gray       93  
      Hazel      64  
      Green      64  
      dtype: int64
```

Crear una Series pasando los datos y los índices como parámetros.

Series

Propiedades de Series

```
[42]: eye = pd.Series({"Brown": 220, "Blue": 215, "Hazel": 93, "Green": 64},  
                      name="eye_color")
```

eye

```
[42]: Brown    220  
      Blue    215  
      Hazel    93  
      Green    64  
      Name: eye_color, dtype: int64
```

Podemos asignar un **nombre** a una Series.

Series

Propiedades de Series

```
[43]: eye.sort_values()
```

```
[43]: Green      64  
      Hazel     93  
      Blue     215  
      Brown    220  
      Name: eye_color, dtype: int64
```

```
[44]: eye.sort_index()
```

```
[44]: Blue      215  
      Brown    220  
      Green     64  
      Hazel     93  
      Name: eye_color, dtype: int64
```

Ordenar una Series por valores o por índice.

Series

Propiedades de Series

```
[45]: eye.unique()
```

```
[45]: array([220, 215,  93,  64])
```

```
[46]: eye.nunique()
```

```
[46]: 4
```

Valores únicos no repetidos (similar a un set) y número de valores únicos.

Series

Propiedades de Series

```
[56]: eye.value_counts()

[56]: 64      2
      220     1
      215     1
      93      1
      Name: count, dtype: int64
```

El método `value_counts` devuelve una Series con la cantidad de elementos de cada tipo.

Series

Indexado y Slicing

```
[58]: ser = pd.Series([0,10,20,30,40], index=["a","b","c","d","e"])
```

```
[60]: ser["b"]
```

```
[60]: np.int64(10)
```

```
[61]: ser[1:3]
```

```
[61]: b    10  
     c    20  
     dtype: int64
```

El **indexado** es por índice y por posición (*por posición está descontinuado, pero hay alternativa*).

El **slicing** funciona por posición.

Series

Indexado y Slicing

```
[62]: ser["b":"d"]
```

```
[62]: b    10  
      c    20  
      d    30  
      dtype: int64
```

```
[63]: ser.index
```

```
[63]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

Slicing por **índice** (sigue el orden del índice de la Series).

Cuidado: con índices no numéricos no excluye el último elemento (no puede hacer `"d"-1`).

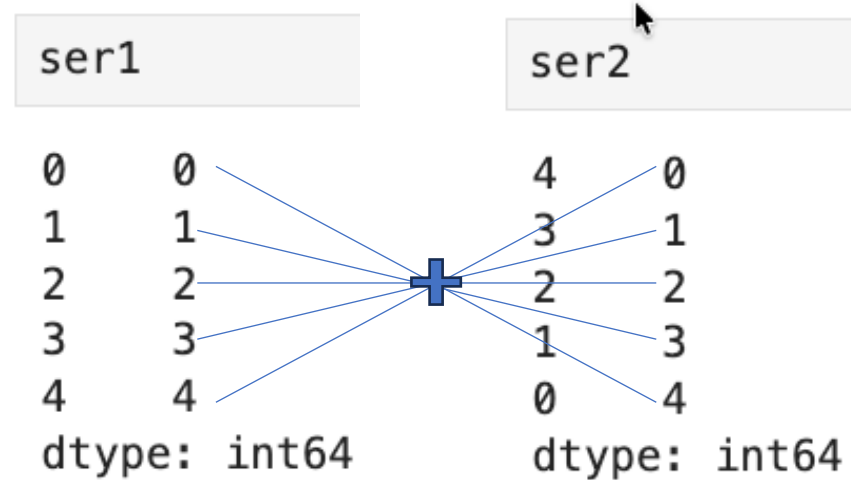
Series

Operaciones aritméticas

```
[66]: ser1 = pd.Series([0,1,2,3,4], index=[0,1,2,3,4])  
      ser2 = pd.Series([0,1,2,3,4], index=[4,3,2,1,0])
```

```
[67]: ser1 + ser2
```

```
[67]: 0      4  
      1      4  
      2      4  
      3      4  
      4      4  
      dtype: int64
```



Al operar con Series **aline**a los índices

Series

Operaciones aritméticas

```
ser1-ser2
```

```
0    -4
```

```
1    -2
```

```
2     0
```

```
3     2
```

```
4     4
```

```
dtype: int64
```

ser1	ser2	ser1-ser2
0	4	-4
1	3	-2
2	2	0
3	1	2
4	0	4

Series

Operaciones aritméticas

```
ser1*ser2
```

```
0    0
1    3
2    4
3    3
4    0
dtype: int64
```

ser1	ser2	ser1*ser2
0	4	0
1	3	3
2	2	4
3	1	3
4	0	0

Series

Operaciones aritméticas

```
ser1/ser2
```

```
0    0.000000
1    0.333333
2    1.000000
3    3.000000
4           inf
dtype: float64
```

ser1	ser2	ser1/ser2
0	4	0.0
1	3	0.33333
2	2	1.0
3	1	3.0
4	0	inf

Series

Operaciones estadísticas

```
[76]: ser1.sum()
```

```
[76]: np.int64(10)
```

```
[77]: ser1.mean()
```

```
[77]: np.float64(2.0)
```

```
[78]: ser1.median()
```

```
[78]: np.float64(2.0)
```

```
[79]: ser1.max()
```

```
[79]: np.int64(4)
```

```
[80]: ser1.min()
```

```
[80]: np.int64(0)
```

```
[81]: ser1.std()
```

```
[81]: np.float64(1.5811388300841898)
```

Series

Transformaciones de Series

```
[82]: ser = pd.Series([160,170,180], name="height")
```

```
[83]: ser.apply(lambda x: x+1000)
```

```
[83]: 0    1160  
      1    1170  
      2    1180  
      Name: height, dtype: int64
```

El método `apply` aplica una función a todos los elementos de una Series.

El método puede ser anónimo (`lambda`) o con nombre si lo creaste previamente.

Índice



1. Introducción

2. Series

3. DataFrames

4. Combinar DataFrames

5. Agrupar DataFrames

6. Preprocesado de datos

DataFrames

Introducción

	Name	Age	Sex
0	Rafa Nadal	35	male
1	Pau Gasol	42	male
2	Ana Peleteiro	19	female

Un **DataFrame** es una **estructura de datos bidimensional** que almacena diversos tipos de datos (cadenas, enteros, reales, datos categóricos, etc.) en columnas.

Cada tabla se compone de **columnas etiquetadas** con un nombre. En el caso del ejemplo son: "Name", "Age" y "Sex."

La columna "Name" es de tipo textual, "Age" es de tipo entero y "Sex" de tipo textual.

La representación es muy similar a la de las **hojas de cálculo**.

DataFrames

Creando un DataFrame

```
[25]: datos = {  
    "Name": ["Rafa Nadal", "Pau Gasol", "Ana Peleteiro"],  
    "Age": [35, 42, 19],  
    "Sex": ["male", "male", "female"]  
}
```

```
[26]: pd.DataFrame(datos)
```

```
[26]:
```

	Name	Age	Sex
0	Rafa Nadal	35	male
1	Pau Gasol	42	male
2	Ana Peleteiro	19	female

Puedes crear un **DataFrame** a partir de un diccionario donde:

- Las **claves** son los nombres de las **columnas**
- Los **valores** son listas con el **contenido** de cada columna

DataFrames

Creando un DataFrame

```
[28]: df = pd.DataFrame({  
    "Name": ["Rafa Nadal", "Pau Gasol", "Ana Peleteiro"],  
    "Age": [35, 42, 19],  
    "Sex": ["male", "male", "female"]  
})
```

```
[30]: df["Age"]
```

```
[30]: 0    35  
     1    42  
     2    19  
      Name: Age, dtype: int64
```

```
[31]: type(df["Age"])
```

```
[31]: pandas.core.series.Series
```

Podemos recuperar una **columna** de un DataFrame mediante el operador **corchete** (`[]`) de indexado (similar a diccionarios de Python).

Las columnas de un DataFrame son **Series**.

DataFrames

Creando un DataFrame

```
[39]: df["Age"][0]
```

```
[39]: np.int64(35)
```

```
[40]: print(df["Age"][0])
```

35

- `df["Age"]` devuelve una Series (columna del DataFrame).
- `[0]` selecciona el primer valor de esa Series.
- El resultado no es un `int` de Python, sino un objeto NumPy (`np.int64`).
- Al imprimirlo aparece como 35,
- Pero el tipo real es `numpy.int64`.

DataFrames

Creando un DataFrame

```
[90]: pd.read_csv("data_iris.csv")
```

```
[90]:
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows x 5 columns

- Podemos cargar un DataFrame a partir de un fichero CSV (valores separados por comas)

DataFrames

Propiedades de un DataFrame

```
[92]: df = pd.read_csv("data_iris.csv")
```

```
[93]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype    
---  ---            -  
0   Sepal.Length    150 non-null   float64  
1   Sepal.Width     150 non-null   float64  
2   Petal.Length    150 non-null   float64  
3   Petal.Width     150 non-null   float64  
4   Species         150 non-null   object   
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB
```

- El método `info` da información sobre el DataFrame: *tipo, índice, columnas, uso de memoria* y, para cada columna, su *nombre, tipo de datos y número de elementos **no nulos***.

DataFrames

Propiedades de un DataFrame

- El método `head` muestra los primeros 5 elementos del DataFrame.
- El método `tail` muestra los últimos 5 elementos del DataFrame.
- Una buena forma de examinar DataFrames grandes sin imprimirlo entero.

```
[94]: df.head()
```

```
[94]:
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
[95]: df.tail()
```

```
[95]:
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

DataFrames

Propiedades de un DataFrame

- Obtener los nombres de **columnas** de un DataFrame

```
[96]: df.columns
```

```
[96]: Index(['Sepal.Length', 'Sepal.Width', 'Petal.Length', 'Petal.Width',  
          'Species'],  
          dtype='object')
```

```
[97]: df.index
```

```
[97]: RangeIndex(start=0, stop=150, step=1)
```

- Obtener los **índices** (filas) de un DataFrame

```
[102]: df.columns = ["sepal_length", "sepal_width", "petal_length", "petal_width", "species"]
```

```
[103]: df.head()
```

- **Cambiar** los nombres de columnas de un DataFrame

```
[103]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

DataFrames

Indexing y Slicing

```
[105]: df[["sepal_length", "sepal_width"]].head()
```

```
[105]:
```

	sepal_length	sepal_width
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6

- Obtener un **subconjunto** de columnas
- Es una **lista** de columnas
- No importa el **orden**

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[107]: exam_data = {  
    'name': ['Ana Pascual', 'Luis Martínez', 'Carmen López'],  
    'math': [90, 80, 70],  
    'eng': [98, 89, 95],  
    'music': [85, 95, 100],  
    'phys': [100, 90, 90]  
}  
  
df = pd.DataFrame(exam_data)  
df
```

```
[107]:
```

	name	math	eng	music	phys
0	Ana Pascual	90	98	85	100
1	Luis Martínez	80	89	95	90
2	Carmen López	70	95	100	90

Creamos un DataFrame de ejemplo

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[108]: df.set_index("name", inplace=True)  
df
```

```
[108]:
```

	math	eng	music	phys
name				
Ana Pascual	90	98	85	100
Luis Martínez	80	89	95	90
Carmen López	70	95	100	90

- Designamos un índice a partir de una columna
- El parámetro `inplace` modifica el DataFrame en lugar de devolver uno nuevo

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[108]:
```

	math	eng	music	phys
name				
Ana Pascual	90	98	85	100
Luis Martínez	80	89	95	90
Carmen López	70	95	100	90

```
[109]: df.loc["Ana Pascual", "eng"]
```

```
[109]: np.int64(98)
```

- El operador `loc[]` permite seleccionar un elemento por nombre de fila (índice) y nombre de columna.

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[110]: df.loc["Ana Pascual", ["eng", "phys"]]
```

```
[110]: eng      98  
      phys    100  
      Name: Ana Pascual, dtype: int64
```

- Puedes seleccionar a la vez varias columnas de una misma fila
- Devolverá una Series con el nombre de la fila y los nombres de columna en el índice

DataFrames

Indexing y Slicing. Operadores loc y iloc

[112]:

		0	1	2	3
	name	math	eng	music	phys
0	Ana Pascual	90	98	85	100
1	Luis Martínez	80	89	95	90
2	Carmen López	70	95	100	90

```
[111]: df.iloc[0, [1,3]]
```

```
[111]: eng      98
      phys    100
      Name: Ana Pascual, dtype: int64
```

- El operador `iloc[]` funciona del mismo modo que `loc[]`, pero utilizando los índices (posiciones) tanto de la fila como de las columnas.

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[113]: df.loc["Ana Pascual", "eng":"phys"]
```

```
[113]: eng      98  
      music    85  
      phys    100  
      Name: Ana Pascual, dtype: int64
```

```
[115]: df.iloc[0, 1:4]
```

```
[115]: eng      98  
      music    85  
      phys    100  
      Name: Ana Pascual, dtype: int64
```

- Ambos operadores permiten el uso del **slicing** para seleccionar **rangos** de **columnas**

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[116]: df.loc[["Ana Pascual", "Carmen López"], ["eng", "phys"]]
```

```
[116]:
```

	eng	phys
name		
Ana Pascual	98	100
Carmen López	95	90

```
[117]: df.iloc[[0,2], [1, 3]]
```

```
[117]:
```

	eng	phys
name		
Ana Pascual	98	100
Carmen López	95	90

- La selección de **múltiples columnas** se puede utilizar también para **múltiples filas**.
- Si selecciones múltiples filas y columnas devolverá un nuevo **DataFrame**.

DataFrames

Indexing y Slicing. Operadores loc y iloc

```
[118]: df.loc["Ana Pascual":"Carmen López", "eng":"phys"]
```

```
[118]:
```

	eng	music	phys
name			
Ana Pascual	98	85	100
Luis Martínez	89	95	90
Carmen López	95	100	90

```
[120]: df.iloc[0:3, 1:4]
```

```
[120]:
```

	eng	music	phys
name			
Ana Pascual	98	85	100
Luis Martínez	89	95	90
Carmen López	95	100	90

El **slicing** también funciona para las **filas**, del mismo modo que para **columnas**.

DataFrames

Índices

```
[121]: df
```

```
[121]:
```

	math	eng	music	phys
name				
Ana Pascual	90	98	85	100
Luis Martínez	80	89	95	90
Carmen López	70	95	100	90

```
[122]: df.reset_index()
```

```
[122]:
```

	name	math	eng	music	phys
0	Ana Pascual	90	98	85	100
1	Luis Martínez	80	89	95	90
2	Carmen López	70	95	100	90

Es posible resetear el índice con el método **reset_index**.

El antiguo índice es restaurado como columna.

DataFrames

Índices

```
[129]: df.index = ["stu1", "stu2", "stu3"]
```

```
[130]: df
```

```
[130]:
```

	name	math	eng	music	phys
stu1	Ana Pascual	90	98	85	100
stu2	Luis Martínez	80	89	95	90
stu3	Carmen López	70	95	100	90

Sustituir el índice asignando una nueva lista con los índices

DataFrames

Índices

```
[133]: df.rename(index={"stu1": "student1"}, columns={"phys": "physics"})
```

```
[133]:
```

	name	math	eng	music	physics
student1	Ana Pascual	90	98	85	100
stu2	Luis Martínez	80	89	95	90
stu3	Carmen López	70	95	100	90

Renombrar filas o columnas con el método **rename**.

Admite los parámetros **index** y **columns**.

Admite un diccionario, sustituyendo la clave por el valor.

DataFrames

Índices

```
[166]: df = df.set_index(["math", "music"])
df
```

```
[166]:
```

		name	eng	phys	spa
math	music				
90.0	85.0	Ana Pascual	98.0	60.0	70.0
80.0	95.0	Luis Martínez	89.0	90.0	80.0
100.0	100.0	Carmen López	95.0	90.0	80.0
87.0	99.0	Pedro Pascal	91.0	100.0	76.0

```
[167]: df.index
```

```
[167]: MultiIndex([( 90.0,  85.0),
                    ( 80.0,  95.0),
                    (100.0, 100.0),
                    ( 87.0,  99.0)],
                  names=['math', 'music'])
```

Con `set_index` podemos crear **Multi-Índices**

DataFrames

Índices

```
[173]: df.reindex(["stu1", "stu2", "stu3", "stu4", "stu5", "stu6"])
```

```
[173]:
```

	math	music	name	eng	phys	spa
stu1	90.0	85.0	Ana Pascual	98.0	60.0	70.0
stu2	80.0	95.0	Luis Martínez	89.0	90.0	80.0
stu3	100.0	100.0	Carmen López	95.0	90.0	80.0
stu4	87.0	99.0	Pedro Pascal	91.0	100.0	76.0
stu5	NaN	NaN	NaN	NaN	NaN	NaN
stu6	NaN	NaN	NaN	NaN	NaN	NaN

- El método `reindex` permite cambiar el índice de un DataFrame
- Si no existía el índice la fila se creará con **NaN**

DataFrames

Índices

```
[175]: df.reindex(["stu1", "stu2", "stu3", "stu4", "stu5", "stu6"], fill_value=0)
```

```
[175]:
```

	math	music	name	eng	phys	spa
stu1	90.0	85.0	Ana Pascual	98.0	60.0	70.0
stu2	80.0	95.0	Luis Martínez	89.0	90.0	80.0
stu3	100.0	100.0	Carmen López	95.0	90.0	80.0
stu4	87.0	99.0	Pedro Pascal	91.0	100.0	76.0
stu5	0.0	0.0	0	0.0	0.0	0.0
stu6	0.0	0.0	0	0.0	0.0	0.0

Se pueden evitar los **NaN** con el argumento `fill_value`

DataFrames

Multi-Índices jerárquicos

```
[199]: my_header = ["a", "b", "c"]  
my_index_out = ["G1"]*3 + ["G2"]*3  
my_index_in = [1,2,3]*2  
my_index_zipped = list(zip(my_index_out, my_index_in))  
my_index_zipped
```

```
[199]: [('G1', 1), ('G1', 2), ('G1', 3), ('G2', 1), ('G2', 2), ('G2', 3)]
```

```
[202]: my_index = pd.MultiIndex.from_tuples(my_index_zipped)  
pd.DataFrame(data=np.random.randn(6,3), index=my_index, columns=my_header)
```

```
[202]:
```

		a	b	c
G1	1	-0.042744	-0.073866	1.530367
	2	-0.419324	-0.658002	-0.438456
	3	0.304558	1.330560	-0.967151
G2	1	0.468582	-0.271541	-0.369606
	2	0.615549	0.772355	-0.202101
	3	0.964261	0.726916	0.819876

Creando un DataFrame con un Multi-Índice jerárquico.

DataFrames

Borrado de filas y columnas

```
[136]: df.drop("stu1", axis=0)
```

```
[136]:
```

	name	math	eng	music	phys
stu2	Luis Martínez	80	89	95	90
stu3	Carmen López	70	95	100	90

```
[137]: df.drop(["stu1", "stu2"], axis=0)
```

```
[137]:
```

	name	math	eng	music	phys
stu3	Carmen López	70	95	100	90

Eliminación de una o varias filas.

El parámetro **axis=0** indica que se deben borrar filas.

El valor por defecto de **axis** es 0.

DataFrames

Borrado de filas y columnas

```
[138]: df.drop("math", axis=1)
```

```
[138]:
```

	name	eng	music	phys
stu1	Ana Pascual	98	85	100
stu2	Luis Martínez	89	95	90
stu3	Carmen López	95	100	90

```
[139]: df.drop(["math", "music", "phys"], axis=1)
```

```
[139]:
```

	name	eng
stu1	Ana Pascual	98
stu2	Luis Martínez	89
stu3	Carmen López	95

Eliminar columnas funciona con el mismo método `drop`, pero con `axis=1`.

Admite una lista de columnas.

DataFrames

Añadir una columna

```
[141]: df["spa"] = 80  
df
```

```
[141]:
```

	name	math	eng	music	phys	spa
stu1	Ana Pascual	90	98	85	100	80
stu2	Luis Martínez	80	89	95	90	80
stu3	Carmen López	70	95	100	90	80

Si asignamos un valor a un nombre de **columna** no existente se crea esa columna y se les asigna el valor a **todas** las filas.

DataFrames

Añadir una fila

```
[146]: df.loc["stu4"] = 0  
df
```

```
[146]:
```

	name	math	eng	music	phys	spa
stu1	Ana Pascual	90	98	85	100	80
stu2	Luis Martínez	80	89	95	90	80
stu3	Carmen López	70	95	100	90	80
stu4	0	0	0	0	0	0

El operador `loc` o `iloc` nos permite asignar un mismo valor a una fila nueva

DataFrames

Añadir una fila

```
[148]: df.loc["stu4"] = ["Pedro Pascal", 87, 91, 99, 84, 76]  
df
```

```
[148]:
```

	name	math	eng	music	phys	spa
stu1	Ana Pascual	90	98	85	100	80
stu2	Luis Martínez	80	89	95	90	80
stu3	Carmen López	70	95	100	90	80
stu4	Pedro Pascal	87	91	99	84	76

También podemos **añadir** (o **modificar**) la fila entera con una **lista** de valores

DataFrames

Modificar un DataFrame

```
[153]: df.loc["stu4", "phys"] = 100  
df.iloc[2, 1] = 100  
df
```

```
[153]:
```

	name	math	eng	music	phys	spa
stu1	Ana Pascual	90.0	98.0	85.0	100.0	80.0
stu2	Luis Martínez	80.0	89.0	95.0	90.0	80.0
stu3	Carmen López	100.0	95.0	100.0	90.0	80.0
stu4	Pedro Pascal	87.0	91.0	99.0	100.0	76.0

Podemos **modificar** un único dato de un DataFrame introduciendo sus “**coordenadas**” a los operadores `loc` o `iloc`.

DataFrames

Modificar un DataFrame

```
[154]: df.loc["stu1", ["phys", "spa"]] = 60, 70
df
```

```
[154]:
```

	name	math	eng	music	phys	spa
stu1	Ana Pascual	90.0	98.0	85.0	60.0	70.0
stu2	Luis Martínez	80.0	89.0	95.0	90.0	80.0
stu3	Carmen López	100.0	95.0	100.0	90.0	80.0
stu4	Pedro Pascal	87.0	91.0	99.0	100.0	76.0

Esta forma de **asignación** funciona con coordenadas “**múltiples**” para modificar diferentes filas o columnas

DataFrames

Transponer un DataFrame

```
[156]: df.transpose()
```

```
[156]:
```

	stu1	stu2	stu3	stu4
name	Ana Pascual	Luis Martínez	Carmen López	Pedro Pascal
math	90.0	80.0	100.0	87.0
eng	98.0	89.0	95.0	91.0
music	85.0	95.0	100.0	99.0
phys	60.0	90.0	90.0	100.0
spa	70.0	80.0	80.0	76.0

Transponer un DataFrame es exactamente igual que con arrays de **NumPy**.

DataFrames

Ordenar un DataFrame

```
[178]: df.sort_values(by="eng", ascending=False)
```

```
[178]:
```

	name	eng	math	music	phys	spa
stu1	Ana Pascual	98.0	90.0	85.0	60.0	70.0
stu3	Carmen López	95.0	100.0	100.0	90.0	80.0
stu4	Pedro Pascal	91.0	87.0	99.0	100.0	76.0
stu2	Luis Martínez	89.0	80.0	95.0	90.0	80.0

- Se puede **ordenar** un DataFrame por sus valores
- Hay que seleccionar la columna para ordenar con el argumento **by**
- Se puede hacer **ascendente** o **descendente**

DataFrames

Ordenar un DataFrame

```
[181]: df.sort_values(by=["spa", "eng"], ascending=False)
```

```
[181]:
```

	name	eng	math	music	phys	spa
stu3	Carmen López	95.0	100.0	100.0	90.0	80.0
stu2	Luis Martínez	89.0	80.0	95.0	90.0	80.0
stu4	Pedro Pascal	91.0	87.0	99.0	100.0	76.0
stu1	Ana Pascual	98.0	90.0	85.0	60.0	70.0

Es posible ordenar por más de una columna

DataFrames

Ordenar un DataFrame

```
[182]: df.sort_index(ascending=False)
```

```
[182]:
```

	name	eng	math	music	phys	spa
stu4	Pedro Pascal	91.0	87.0	99.0	100.0	76.0
stu3	Carmen López	95.0	100.0	100.0	90.0	80.0
stu2	Luis Martínez	89.0	80.0	95.0	90.0	80.0
stu1	Ana Pascual	98.0	90.0	85.0	60.0	70.0

Se puede ordenar por **índice**

Índice



1. Introducción

2. Series

3. DataFrames

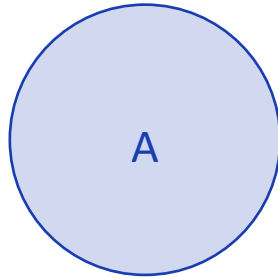
4. Combinar DataFrames

5. Agrupar DataFrames

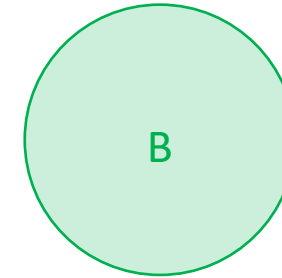
6. Preprocesado de datos

Combinar DataFrames

Fusión de DataFrames



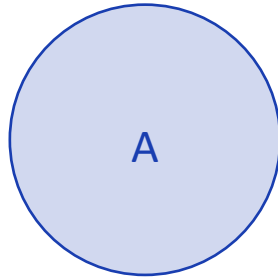
Name	Gender	Age
Harry Potter	Male	23
David Baker	Male	31
John Smith	Male	22
Juan Martinez	Male	36
Jane Connor	Female	30



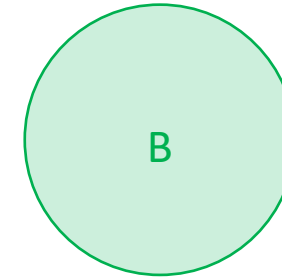
Name	Position	Wage
John Smith	Intern	25000
Alex Du Bois	Team Lead	75000
Joanne Rowling	Manager	90000
Jane Connor	Manager	70000

Combinar DataFrames

Fusión de DataFrames



Name	Gender	Age
Harry Potter	Male	23
David Baker	Male	31
John Smith	Male	22
Juan Martinez	Male	36
Jane Connor	Female	30

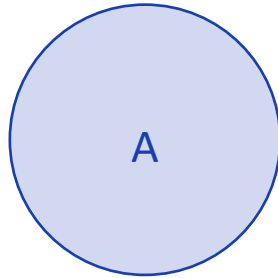


Name	Position	Wage
John Smith	Intern	25000
Alex Du Bois	Team Lead	75000
Joanne Rowling	Manager	90000
Jane Connor	Manager	70000

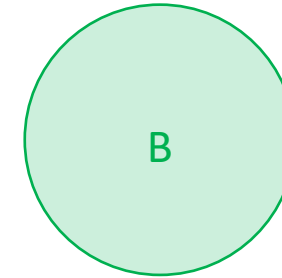
El campo **Name** de **A** y **B** se utiliza como clave para **fusionar** los DataFrames

Combinar DataFrames

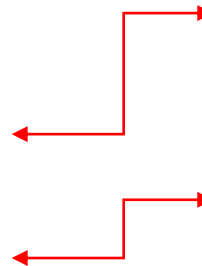
Fusión de DataFrames



Name	Gender	Age
Harry Potter	Male	23
David Baker	Male	31
John Smith	Male	22
Juan Martinez	Male	36
Jane Connor	Female	30



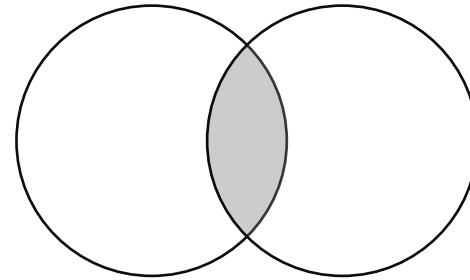
Name	Position	Wage
John Smith	Intern	25000
Alex Du Bois	Team Lead	75000
Joanne Rowling	Manager	90000
Jane Connor	Manager	70000



Inner Join: únicamente se fusionan los campos con clave en común

Combinar DataFrames

Fusión de DataFrames

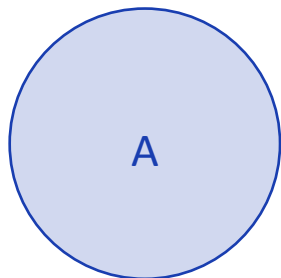


A.Name	Gender	Age	B.Name	Position	Wage
John Smith	Male	22	John Smith	Intern	25000
Jane Connor	Female	30	Jane Connor	Manager	70000

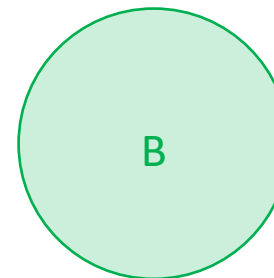
Inner Join: únicamente se fusionan los campos con clave en común

Combinar DataFrames

Fusión de DataFrames



Name	Gender	Age
Harry Potter	Male	23
David Baker	Male	31
John Smith	Male	22
Juan Martinez	Male	36
Jane Connor	Female	30

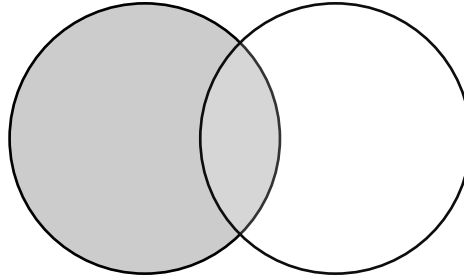


Name	Position	Wage
John Smith	Intern	25000
Alex Du Bois	Team Lead	75000
Joanne Rowling	Manager	90000
Jane Connor	Manager	70000

Left Join: Todos los campos de **A** se mantienen, únicamente se fusionan los de **B** para los que coincide la clave

Combinar DataFrames

Fusión de DataFrames

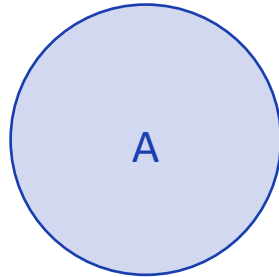


A.Name	Gender	Age	B.Name	Position	Wage
Harry Potter	Male	23	NA	NA	NA
David Baker	Male	31	NA	NA	NA
John Smith	Male	22	John Smith	Intern	25000
Juan Martinez	Male	36	NA	NA	NA
Jane Connor	Female	30	Jane Connor	Manager	70000

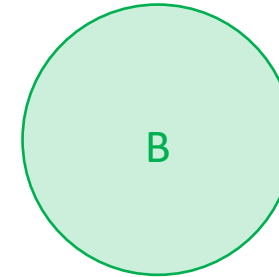
Left Join: Los campos de **A** que no existen en **B** generarán datos indeterminados.

Combinar DataFrames

Fusión de DataFrames



Name	Gender	Age
Harry Potter	Male	23
David Baker	Male	31
John Smith	Male	22
Juan Martinez	Male	36
Jane Connor	Female	30

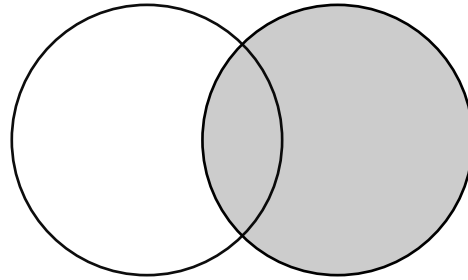


Name	Position	Wage
John Smith	Intern	25000
Alex Du Bois	Team Lead	75000
Joanne Rowling	Manager	90000
Jane Connor	Manager	70000

Right Join: Todos los campos de **B** se mantienen, únicamente se fusionan los de **A** para los que coincide la clave

Combinar DataFrames

Fusión de DataFrames

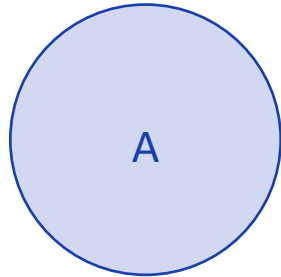


A.Name	Gender	Age	B.Name	Position	Wage
John Smith	Male	22	John Smith	Intern	25000
NA	NA	NA	Alex Du Bois	Team Lead	75000
NA	NA	NA	Joanne Rowling	Manager	90000
Jane Connor	Female	30	Jane Connor	Manager	70000

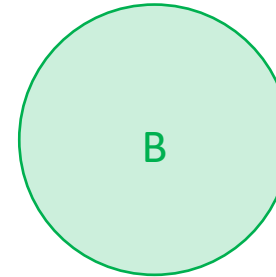
Right Join: Los campos de **B** que no existen en **A** generarán datos indeterminados.

Combinar DataFrames

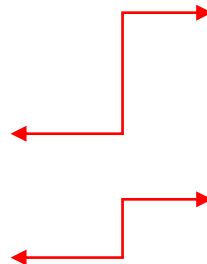
Fusión de DataFrames



Name	Gender	Age
Harry Potter	Male	23
David Baker	Male	31
John Smith	Male	22
Juan Martinez	Male	36
Jane Connor	Female	30



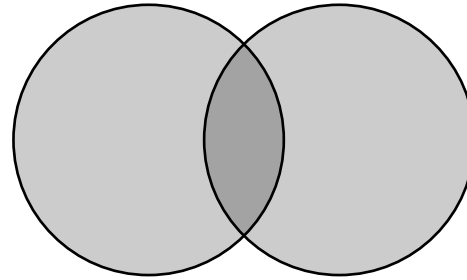
Name	Position	Wage
John Smith	Intern	25000
Alex Du Bois	Team Lead	75000
Joanne Rowling	Manager	90000
Jane Connor	Manager	70000



Full Outer Join: Se mantienen todos los campos de **A** y **B**.

Combinar DataFrames

Fusión de DataFrames



A.Name	Gender	Age	B.Name	Position	Wage
Harry Potter	Male	23	NA	NA	NA
David Baker	Male	31	NA	NA	NA
John Smith	Male	22	John Smith	Intern	25000
Juan Martinez	Male	36	NA	NA	NA
Jane Connor	Female	30	Jane Connor	Manager	70000
NA	NA	NA	Alex Du Bois	Team Lead	75000
NA	NA	NA	Joanne Rowling	Manager	90000

Full Outer Join: Es posible que se generen datos indeterminados en campos de ambos DataFrames.

Combinar DataFrames

Fusión de DataFrames

```
[ ]: pd.merge(df_left, df_right, on="NAME")
      pd.merge(df_left, df_right, on="NAME", how="inner")
      pd.merge(df_left, df_right, on="NAME", how="left")
      pd.merge(df_left, df_right, on="NAME", how="right")
      pd.merge(df_left, df_right, on="NAME", how="outer")
```

- El método `pd.merge` fusiona dos DataFrames
- El campo `on` permite indicar qué columna utilizar para enlazar las filas
- El campo `how` permite indicar el método de fusión. Por defecto es `"inner"`.

Combinar DataFrames

Fusión de DataFrames

```
[ ]: pd.merge(df_left, df_right, right_on="NAME", left_on="Nombre", how="inner")
      pd.merge(df_left, df_right, right_on="NAME", left_on="Nombre", how="left")
      pd.merge(df_left, df_right, right_on="NAME", left_on="Nombre", how="right")
      pd.merge(df_left, df_right, right_on="NAME", left_on="Nombre", how="outer")
```

- Si el campo clave en el DataFrame izquierdo y derecho tiene un nombre que difiere podemos utilizar los argumentos `left_on` y `right_on`.
- Estos argumentos sustituyen al argumento `on`.

Combinar DataFrames

Concatenación de DataFrames

```
[183]: df_A = pd.DataFrame({"c1": [10,20,30], "c2": [9,8,7], "c3": [1,2,3]})  
      df_B = pd.DataFrame({"c1": [40,50,60], "c2": [6,5,4], "c3": [4,5,6]})
```

```
[186]: pd.concat([df_A, df_B], axis=0)
```

```
[186]:
```

	c1	c2	c3
0	10	9	1
1	20	8	2
2	30	7	3
0	40	6	4
1	50	5	5
2	60	4	6

- Otra forma de fusionar DataFrames es **concatenarlos**.
- El método `pd.concat` concatena en vertical (`axis=0`) por defecto.
- Los **índices** se mantienen y pueden **repetirse**

Combinar DataFrames

Concatenación de DataFrames

```
[187]: pd.concat([df_A, df_B], axis=1)
```

```
[187]:
```

	c1	c2	c3	c1	c2	c3
0	10	9	1	40	6	4
1	20	8	2	50	5	5
2	30	7	3	60	4	6

- La concatenación también puede hacerse en horizontal (**axis=1**)
- En este caso se alinean los **índices** pero pueden repetirse los nombres de **columnas**.

Combinar DataFrames

Concatenación de DataFrames

```
[189]: df_A = pd.DataFrame({"a": ["a1","a2","a3"], "b": ["b1","b2","b3"], "c": ["c1","c2","c3"]})  
df_A.index = [1,2,3]  
df_B = pd.DataFrame({"b": ["b1","b2","b3"], "c": ["c1","c2","c3"], "d": ["d1","d2","d3"]})  
df_B.index = [2,3,4]
```

```
pd.concat([df_A, df_B], axis=0)
```

[191]:

	a	b	c	d
1	a1	b1	c1	NaN
2	a2	b2	c2	NaN
3	a3	b3	c3	NaN
2	NaN	b1	c1	d1
3	NaN	b2	c2	d2
4	NaN	b3	c3	d3

```
pd.concat([df_A, df_B], axis=1)
```

[192]:

	a	b	c	b	c	d
1	a1	b1	c1	NaN	NaN	NaN
2	a2	b2	c2	b1	c1	d1
3	a3	b3	c3	b2	c2	d2
4	NaN	NaN	NaN	b3	c3	d3

- Al concatenar en horizontal o vertical pueden surgir **indeterminados** si no existe la columna o el índice correspondiente.
- Se rellenan con **NaN**.

Combinar DataFrames

Concatenación de DataFrames

```
[195]: pd.concat([df_A, df_B], axis=0, ignore_index=True)
```

```
[195]:
```

	a	b	c	d
0	a1	b1	c1	NaN
1	a2	b2	c2	NaN
2	a3	b3	c3	NaN
3	NaN	b1	c1	d1
4	NaN	b2	c2	d2
5	NaN	b3	c3	d3

```
pd.concat([df_A, df_B], axis=0)
```

```
[191]:
```

	a	b	c	d
1	a1	b1	c1	NaN
2	a2	b2	c2	NaN
3	a3	b3	c3	NaN
2	NaN	b1	c1	d1
3	NaN	b2	c2	d2
4	NaN	b3	c3	d3

- Se puede **ignorar los índices** originales en la concatenación vertical.
- También sería válido realizar un `reset_index()` a posteriori.

Combinar DataFrames

Ejemplo

```
In [11]: df1.head()
```

```
Out[11]:
```

	id	stock_name	value	price
0	128940	Hanmi Pharmaceutical	59385.666667	421000
1	130960	CJ E&M	58540.666667	98900
2	138250	NS Shopping	14558.666667	13200
3	139480	E-mart	239230.833333	254500
4	142280	Green Cross Medical Science Corporation	468.833333	10200

[1] Cargamos un dataset con datos de acciones del mercado de valores.

Combinar DataFrames

Ejemplo

In [12]: `df2.head()`

Out[12]:

	id	name	eps	bps	per	pbr
0	130960	CJ E&M	6301.333333	54068	15.695091	1.829178
1	136480	Harim Co.	274.166667	3551	11.489362	0.887074
2	138040	Meritz Financial Group	2122.333333	14894	6.313806	0.899691
3	139480	E-mart	18268.166667	295780	13.931338	0.860437
4	145990	Samyang	5741.000000	108090	14.283226	0.758627

[2] Cargamos un segundo dataset con información adicional sobre acciones.

Combinar DataFrames

Ejemplo

```
In [13]: pd.merge(df1, df2)
```

```
Out[13]:
```

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068	15.695091	1.829178
1	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780	13.931338	0.860437
2	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090	14.283226	0.758627
3	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
4	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335	40.802348	0.651359

[3] Al intentar fusionarlos descubrimos que al unir por el campo **id** y método **inner** se pierden muchas filas.

Combinar DataFrames

Ejemplo

```
In [21]: df1 = pd.read_excel('./stock price.xlsx')  
df2 = pd.read_excel('./stock valuation.xlsx')
```

```
In [22]: pd.merge(df1, df2, on='id', how='left')
```

Out[22]:

	id	stock_name	value	price	name	eps	bps	per	pbr
0	128940	Hanmi Pharmaceutical	59385.666667	421000	NaN	NaN	NaN	NaN	NaN
1	130960	CJ E&M	58540.666667	98900	CJ E&M	6301.333333	54068.0	15.695091	1.829178
2	138250	NS Shopping	14558.666667	13200	NaN	NaN	NaN	NaN	NaN
3	139480	E-mart	239230.833333	254500	E-mart	18268.166667	295780.0	13.931338	0.860437
4	142280	Green Cross Medical Science Corporation	468.833333	10200	NaN	NaN	NaN	NaN	NaN
5	145990	Samyang	82750.000000	82000	Samyang	5741.000000	108090.0	14.283226	0.758627
6	185750	Chong Kun Dang	40293.666667	100500	Chong Kun Dang	3990.333333	40684.0	25.185866	2.470259
7	192400	Cuckoo Holdings	179204.666667	177500	NaN	NaN	NaN	NaN	NaN
8	199800	Toolgen	-2514.333333	115400	NaN	NaN	NaN	NaN	NaN
9	204210	Mode Tour Reit	3093.333333	3475	Mode Tour Reit	85.166667	5335.0	40.802348	0.651359

[4] Si utilizamos el método **left** tendremos muchas columnas con **NaN** a la derecha.

Combinar DataFrames

Ejemplo

```
In [23]: pd.merge(df1, df2, on='id', how='right')
```

```
Out[23]:
```

	id	stock_name	value	price	name	eps	bps	per	pbr
0	130960	CJ E&M	58540.666667	98900.0	CJ E&M	6301.333333	54068	15.695091	1.829178
1	136480	NaN	NaN	NaN	Harim Co.	274.166667	3551	11.489362	0.887074
2	138040	NaN	NaN	NaN	Meritz Financial Group	2122.333333	14894	6.313806	0.899691
3	139480	E-mart	239230.833333	254500.0	E-mart	18268.166667	295780	13.931338	0.860437
4	145990	Samyang	82750.000000	82000.0	Samyang	5741.000000	108090	14.283226	0.758627
5	161390	NaN	NaN	NaN	Hankook Tire	5648.500000	51341	7.453306	0.820007
6	181710	NaN	NaN	NaN	NHN Entertainment	2110.166667	78434	30.755864	0.827447
7	185750	Chong Kun Dang	40293.666667	100500.0	Chong Kun Dang	3990.333333	40684	25.185866	2.470259
8	204210	Mode Tour Reit	3093.333333	3475.0	Mode Tour Reit	85.166667	5335	40.802348	0.651359
9	207940	NaN	NaN	NaN	Samsung BioLogics	4644.166667	60099	89.790059	6.938551

[5] Si utilizamos el método **right** tendremos el mismo problema a la izquierda.

Combinar DataFrames

Ejemplo

```
In [24]: pd.merge(df1, df2, on='id', how='outer')
```

```
Out[24]:
```

	id	stock_name	value	price	name	eps	bps	per	pbr
0	128940	Hanmi Pharmaceutical	59385.666667	421000.0	NaN	NaN	NaN	NaN	NaN
1	130960	CJ E&M	58540.666667	98900.0	CJ E&M	6301.333333	54068.0	15.695091	1.829178
2	138250	NS Shopping	14558.666667	13200.0	NaN	NaN	NaN	NaN	NaN
3	139480	E-mart	239230.833333	254500.0	E-mart	18268.166667	295780.0	13.931338	0.860437
4	142280	Green Cross Medical Science Corporation	468.833333	10200.0	NaN	NaN	NaN	NaN	NaN
5	145990	Samyang	82750.000000	82000.0	Samyang	5741.000000	108090.0	14.283226	0.758627
6	185750	Chong Kun Dang	40293.666667	100500.0	Chong Kun Dang	3990.333333	40684.0	25.185866	2.470259
7	192400	Cuckoo Holdings	179204.666667	177500.0	NaN	NaN	NaN	NaN	NaN
8	199800	Toolgen	-2514.333333	115400.0	NaN	NaN	NaN	NaN	NaN
9	204210	Mode Tour Reit	3093.333333	3475.0	Mode Tour Reit	85.166667	5335.0	40.802348	0.651359
10	136480	NaN	NaN	NaN	Harim Co.	274.166667	3551.0	11.489362	0.887074
11	138040	NaN	NaN	NaN	Meritz Financial Group	2122.333333	14894.0	6.313806	0.899691
12	161390	NaN	NaN	NaN	Hankook Tire	5648.500000	51341.0	7.453306	0.820007
13	181710	NaN	NaN	NaN	NHN Entertainment	2110.166667	78434.0	30.755864	0.827447
14	207940	NaN	NaN	NaN	Samsung BioLogics	4644.166667	60099.0	89.790059	6.938551

[6] El método **outer** nos deja NaN en ambos lados.

Índice



- 1. Introducción**
- 2. Series**
- 3. DataFrames**
- 4. Combinar DataFrames**
- 5. Agrupar DataFrames**
- 6. Preprocesado de datos**

Agrupar DataFrames

groupby()

```
In [1]: import pandas as pd
```

```
In [2]: df=pd.read_csv('data_studentlist.csv')
```

```
In [3]: df.head(3)
```

```
Out[3]:
```

	name	gender	age	grade	absence	bloodtype	height	weight
0	Jared Diamond	M	23	3	Y	O	165.3	68.2
1	Sarah O'Donnel	F	22	2	N	AB	170.1	53.0
2	Brian Martin	M	24	4	N	B	175.0	80.1

```
In [4]: df.groupby(['gender','bloodtype'])['height'].mean()
```

```
Out[4]:
```

gender	bloodtype	height
F	A	172.450000
	AB	170.100000
	B	158.200000
	O	164.433333
M	A	165.700000
	AB	181.050000
	B	174.550000
	O	166.200000

Name: height, dtype: float64

Podemos agrupar datos con el método **groupby**.

Agrupar DataFrames

groupby()

Función de agregación para el valor

```
df.groupby(['gender', 'bloodtype'])['height'].mean()
```

Columnas seleccionadas como índice

Columnas seleccionadas como valor

gender	bloodtype
F	A
	AB
	B
	O
M	A
	AB
	B
	O

Name: height, dtype: float64

172.450000
170.100000
158.200000
164.433333
165.700000
181.050000
174.550000
166.200000

Agrupar DataFrames

pivot_table()

```
In [7]: my_dict = {"Size": ["L", "L", "M", "M", "M", "S", "S", "S", "S"],
                  "Type": ["A", "A", "A", "B", "B", "A", "A", "B", "B"],
                  "Location": ["L1", "L1", "L1", "L2", "L2", "L1", "L2", "L2", "L1"],
                  "A": [1, 2, 2, 3, 3, 4, 5, 6, 7],
                  "B": [2, 4, 5, 5, 6, 6, 8, 9, 9]}
df = pd.DataFrame(my_dict)
df
```

```
Out[7]:
```

	Size	Type	Location	A	B
0	L	A	L1	1	2
1	L	A	L1	2	4
2	M	A	L1	2	5
3	M	B	L2	3	5
4	M	B	L2	3	6
5	S	A	L1	4	6
6	S	A	L2	5	8
7	S	B	L2	6	9
8	S	B	L1	7	9

Como alternativa a `groupby` podemos usar `pivot_table`

Agrupar DataFrames

`pivot_table()`

```
In [9]: dfr = pd.pivot_table(df, index=['Size', 'Type'], columns='Location', values='B')  
dfr
```

```
Out[9]:
```

		Location	L1	L2
Size	Type			
L	A	3.0	NaN	
	A	5.0	NaN	
M	B	NaN	5.5	
	A	6.0	8.0	
S	B	9.0	9.0	

Se deben indicar:

- las columnas que serán el index (o multi-index)
- la columna que pivotará sus valores en columnas
- la columna que se utilizará como valores

Si no existe un valor para una fila determinada será NaN

Agrupar DataFrames

`pivot_table()`

```
In [10]: dfr.columns
```

```
Out[10]: Index(['L1', 'L2'], dtype='object', name='Location')
```

```
In [11]: dfr.index
```

```
Out[11]: MultiIndex([(L', 'A'),  
                    (M', 'A'),  
                    (M', 'B'),  
                    (S', 'A'),  
                    (S', 'B')],  
                    names=['Size', 'Type'])
```

Se generará un Multi-Index cuando sea necesario

Agrupar DataFrames

`pivot_table()`

```
In [12]: pd.pivot_table(df, index=['Size','Type'], columns='Location', values='B', fill_value=0)
```

Out[12]:

		Location	L1	L2
Size	Type			
L	A	3	0.0	
	A	5	0.0	
M	B	0	5.5	
	A	6	8.0	
S	B	9	9.0	

Out[9]:

		Location	L1	L2
Size	Type			
L	A	3.0	NaN	
	A	5.0	NaN	
M	B	NaN	5.5	
	A	6.0	8.0	
S	B	9.0	9.0	



Con el parámetro `fill_value` evitamos los NaN

Agrupar DataFrames

`pivot_table()`

```
In [14]: import numpy as np
```

```
In [15]: pd.pivot_table(df, index=['Size', 'Type'], columns='Location', values='B', aggfunc = np.mean, fill_value=0)
```

```
Out[15]:
```

		Location	L1	L2
Size	Type			
L	A	3	0.0	
	A	5	0.0	
M	B	0	5.5	
	A	6	8.0	
S	B	9	9.0	

Con el parámetro **aggfunc** podemos definir la función de agregación. Puede ser una función existente (como `np.mean`) o una lambda.

Agrupar DataFrames

`pivot_table()`

```
In [16]: dfr = pd.pivot_table(df, index='Location', columns=['Size','Type'], values='B')  
dfr
```

```
Out[16]:
```

	Size	L			M		S	
	Type	A	A	B	A	B	A	B
Location								
L1		3.0	5.0	NaN	6.0	9.0		
L2		NaN	NaN	5.5	8.0	9.0		

Si seleccionamos una columna para el índice y varias para las columnas veremos el mismo resultado, pero traspuesto

Agrupar DataFrames

`pivot_table()`

```
In [20]: pd.pivot_table(df, index=['Size', 'Type'], values=['A', 'B'], aggfunc=np.mean)
```

Out[20]:

		A	B
Size	Type		
L	A	1.5	3.0
	B	2.0	5.0
M	A	3.0	5.5
	B	4.5	7.0
S	A	6.5	9.0
	B		

Ejemplo agrupando las columnas "A" y "B" con la media
En este caso no utilizamos el parámetro `columns`

Sería equivalente al siguiente `groupby`:

```
In [21]: df.groupby(['Size', 'Type'])[['A', 'B']].mean()
```

Agrupar DataFrames

pivot_table()

```
In [22]: pd.pivot_table(df, index=['Size', 'Type'], values=['A', 'B'], aggfunc={'A':np.max, 'B':np.min})
```

Out[22]:

		A	B
Size	Type		
L	A	2	2
	B	3	5
M	A	5	6
	B	7	9
S	A	2	5
	B	3	5

Es posible definir una función de agregación **diferente** para cada columna pasando un **diccionario** como argumento.

La clave es el nombre de columna y el valor corresponde a la función

Índice

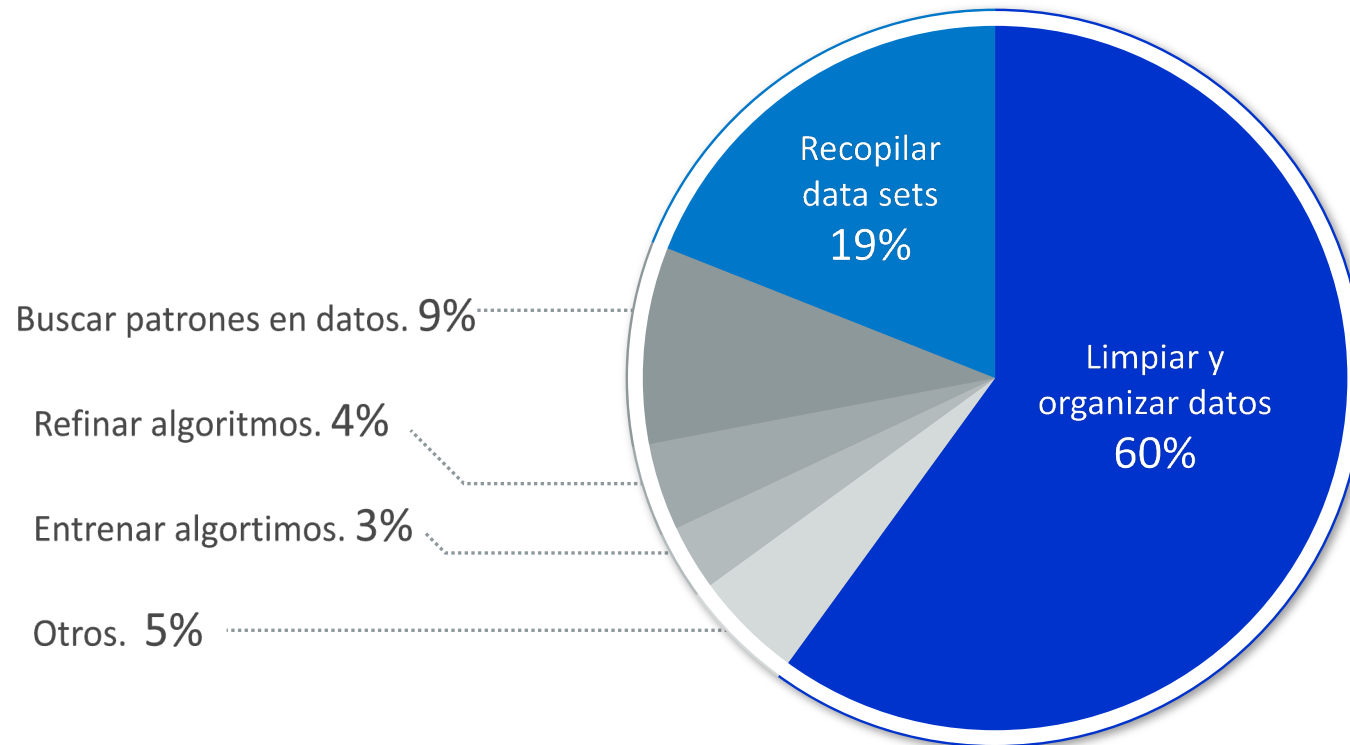


- 1. Introducción**
- 2. Series**
- 3. DataFrames**
- 4. Combinar DataFrames**
- 5. Agrupar DataFrames**
- 6. Preprocesado de datos**

Preprocesado de datos

Introducción

¿A qué dedica más tiempo un científico de datos?

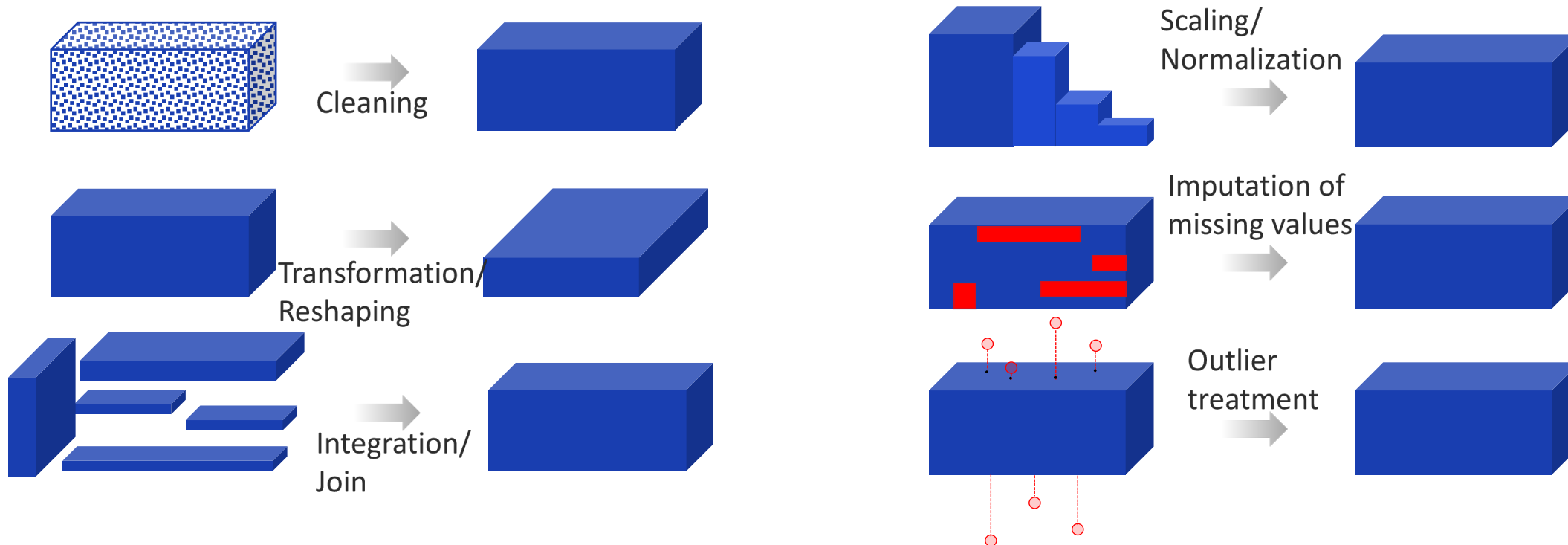


<https://www.forbes.com/sites/gilpress/2016/03/23/data-preparation-most-time-consuming-least-enjoyable-data-science-task-survey-says/#790d18c36f63>

Preprocesado de datos

Preprocesado de datos

Operaciones típicas



Preprocesado de datos

Identificando propiedades de los datos

Clasificación de los datos:

- Los **datos continuos** y los **datos categóricos** son los tipos más básicos de datos estructurados.
- Los datos continuos incluyen datos de **continuidad**, como la velocidad y la duración del viento, y datos **discretos**, como la frecuencia de ocurrencia de eventos.
- Los datos **categóricos** se refieren, por ejemplo, a los nombres de ciudades de cada país (Washington, Nueva York, Los Ángeles) o al tipo de vehículo (autobús, taxi, camión).
- Los datos **binarios** son un caso especial dentro de los categóricos, en los que solo hay dos valores posibles, como 0 y 1, sí/no, o verdadero/falso.
- Dentro de los tipos categóricos, las **valoraciones** (1, 2, 3, 4, 5), en las que los valores de la categoría tienen un orden, se denominan **datos ordinales**.

Preprocesado de datos

Valores ausentes (Missing values)

- Los valores ausentes se refieren a datos que **faltan** o están **vacíos**
- Estos pueden **distorsionar** los resultados del análisis o **impedir** la aplicación de funciones.
- En algunos casos, los valores perdidos generados en una variable no están relacionados con otras variables, por lo que deben eliminarse o sustituirse según la situación.
- Asumiendo que cada variable sigue una distribución de probabilidad específica, se **estiman** los parámetros de dicha distribución y se **reemplazan** los valores perdidos.
- Existen métodos de sustitución como la **media**, la **mediana** o la **moda**.

Preprocesado de datos

Valores ausentes (Missing values)

```
In [1]: 1 import seaborn as sns
        2 df = sns.load_dataset('titanic')
        3 nan_deck = df['deck'].value_counts(dropna=False)
        4 print(nan_deck)
```

NaN	688
-----	-----

C	59
---	----

B	47
---	----

D	33
---	----

E	32
---	----

A	15
---	----

F	13
---	----

G	4
---	---

Name: deck, dtype: int64

Contar los **valores nulos** en una columna

Preprocesado de datos

Valores ausentes (Missing values)

```
In [2]: print(df.head().isnull())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	

	who	adult_male	deck	embark_town	alive	alone
0	False	False	True	False	False	False
1	False	False	False	False	False	False
2	False	False	True	False	False	False
3	False	False	False	False	False	False
4	False	False	True	False	False	False

El método `isnull` devuelve una máscara donde los valores nulos se ponen a **True** y los valores no nulos a **False**.

Preprocesado de datos

Valores ausentes (Missing values)

```
In [3]: print(df.head().notnull())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	\
0	True	True	True	True	True	True	True	True	True	True	
1	True	True	True	True	True	True	True	True	True	True	
2	True	True	True	True	True	True	True	True	True	True	
3	True	True	True	True	True	True	True	True	True	True	
4	True	True	True	True	True	True	True	True	True	True	

	adult_male	deck	embark_town	alive	alone
0	True	False	True	True	True
1	True	True	True	True	True
2	True	False	True	True	True
3	True	True	True	True	True
4	True	False	True	True	True

El método `notnull` devuelve una máscara donde los valores nulos se ponen a **False** y los valores no nulos a **True**.

Preprocesado de datos

Valores ausentes (Missing values)

```
In [4]: print(df.head().isnull().sum(axis=0))
```

```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
deck          3
embark_town   0
alive         0
alone         0
dtype: int64
```

Puedes contar cuantos valores nulos hay por columna.

Preprocesado de datos

Valores ausentes (Missing values)

```
In [5]: df_thresh = df.dropna(axis=1, thresh=500)
print(df_thresh.columns)
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',
       'alone'],
      dtype='object')
```

El método `dropna` elimina las columnas con valores nulos

El parámetro `thresh=500` hace que solo se eliminen las columnas con 500 o más **NaN**

En el caso del ejemplo se ha eliminado la columna **Deck** que tenía 688 **NaN** de 891 valores

Preprocesado de datos

Valores ausentes (Missing values)

```
In [6]: df_age = df.dropna(subset=['age'], how='any', axis=0)
        print(len(df_age))
```

714

Con **subset** puedes limitar a un número de columnas

El parámetro **how="any"** elimina la fila si hay al menos un valor nulo

En el caso del ejemplo se han eliminado las filas que no tienen valor en la columna **Age** (177 valores de 891)

Preprocesado de datos

Reemplazando Valores ausentes

```
In [7]: 1 mean_age = df['age'].mean(axis=0)
        2 df['age'].fillna(mean_age, inplace=True)
        3 print(df['age'].head(10))
```

```
0    22.000000
1    38.000000
2    26.000000
3    35.000000
4    35.000000
5    29.699118
6    54.000000
7     2.000000
8    27.000000
9    14.000000
```

```
Name: age, dtype: float64
```

El método `fillna` rellena los valores ausentes con el valor proporcionado

En el ejemplo se rellenan las filas sin edad con la edad media del dataset

Preprocesado de datos

Reemplazando Valores ausentes

```
In [8]: 1 df = sns.load_dataset('titanic')
        2 print(df['embark_town'][825:830])
        3 print('\n')
```

```
825    Queenstown
826    Southampton
827     Cherbourg
828    Queenstown
829           NaN
Name: embark_town, dtype: object
```

¿Cómo reemplazamos valores cuando son categóricos y no podemos calcular la media?

Preprocesado de datos

Reemplazando Valores ausentes

```
In [9]: most_freq = df['embark_town'].value_counts(dropna=True).idxmax()  
print(most_freq)  
print('\n')
```

Southampton

```
In [10]: df['embark_town'].fillna(most_freq, inplace=True)  
  
print(df['embark_town'][825:830])
```

825 Queenstown

826 Southampton

827 Cherbourg

828 Queenstown

829 Southampton

Name: embark_town, dtype: object

Una buena solución es utilizar el valor **más frecuente**.

Preprocesado de datos

Reemplazando Valores ausentes

```
df = sns.load_dataset('titanic')
df['embark_town'].fillna(method='ffill', inplace=True)
print(df['embark_town'][825:830])
```

825	Queenstown
826	Southampton
827	Cherbourg
828	Queenstown
829	Queenstown

Name: embark_town, dtype: object

Otra opción es utilizar el parámetro `method="ffill"`, que rellena los datos faltantes por los de la fila justo anterior

El criterio para rellenar datos faltantes es el **sentido común** y el conocimiento del **dominio** de los datos.

Preprocesado de datos

Eliminar duplicados

```
In [12]: import pandas as pd
df = pd.DataFrame({'c1': ['a', 'a', 'b', 'a', 'b'],
                  'c2': [1, 1, 1, 2, 2],
                  'c3': [1, 1, 2, 2, 2]})

print(df)
```

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
In [13]: df_dup = df.duplicated()
print(df_dup)
```

0	False
1	True
2	False
3	False
4	False

dtype: bool

Con `duplicated` obtenemos una máscara de qué filas tienen valores duplicados.

La fila 0 y 1 están duplicadas, pero se marca como duplicada la segunda.

Preprocesado de datos

Eliminar duplicados

```
In [14]: col_dup = df['c2'].duplicated()  
print(col_dup)
```

```
0    False  
1     True  
2     True  
3    False  
4     True  
Name: c2, dtype: bool
```

Es posible utilizar `duplicated` para una única columna.

Preprocesado de datos

Eliminar duplicados

```
In [15]: df2 = df.drop_duplicates()
print(df2)
```

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
In [16]: df3 = df.drop_duplicates(subset=['c2', 'c3'])
print(df3)
```

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2

El método `drop_duplicates` elimina las filas duplicadas.
Se puede utilizar con un **subset** de columnas.

Preprocesado de datos

Ingeniería de características

Date		Year	Month	Day	Weekday
2009-8-21	↔	2009	8	21	Friday
2013-8-27		2013	8	27	Tuesday
2019-2-11		2019	2	11	Monday

X_coordinate	Y_coordinate		Distance
3	4	↔	5
5	12		13
9	12		15

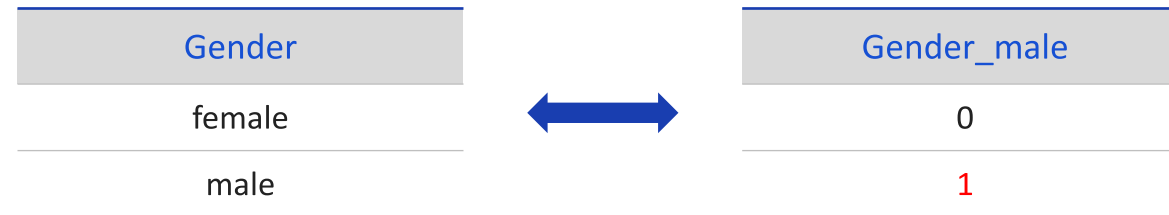
Las variables derivadas se crean a partir de otras variables característica

Preprocesado de datos

Ingeniería de características

Variables categóricas: variables **dummy**

- Una variable **dummy** es una variable derivada que solo puede valer **0 o 1**
- Una variable categórica genera ($\# \text{categorías} - 1$) variables dummy
- **Ejemplo:** La categoría género tiene dos valores, por lo que solo se requiere una variable dummy



Preprocesado de datos

Ingeniería de características

Variables categóricas: variables **dummy**

- Con tres categorías en la variable **Species** son necesarias dos variables **dummy**.

Species		Species_versicolor	Species_virginica
setosa	↔	0	0
versicolor		1	0
virginica		0	1

Preprocesado de datos

Ingeniería de características

One-hot encoding

- Se crean tantas variables como categorías

X0	X1	X2	X3	X4	X5	X6	X7	X8	X9
0	0	0	0	0	1	0	0	0	0

Diagram illustrating One-hot encoding. A blue box with the number 5 has an arrow pointing to the X5 column header. The value 1 in the X5 row is highlighted in red.

- Las variables numéricas se pueden agrupar en intervalos.

Age		Age_10	Age_20	Age_30	Age_40	Age_50	Age_60	Age_70	Age_80
8		0	0	0	0	0	0	0	0
17	↔	1	0	0	0	0	0	0	0
26		0	1	0	0	0	0	0	0
63		0	0	0	0	0	1	0	0

Diagram illustrating numerical variable grouping into intervals. A blue double-headed arrow points from the 'Age' column to the 'Age_10' through 'Age_80' columns. The values 1 and 1 in the Age_10 and Age_20 rows are highlighted in red.

Preprocesado de datos

Ingeniería de características

Ejemplo

```
In [2]: 1 df = pd.read_csv('./auto-mpg.csv', header=None)
2
3 df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
4               'acceleration', 'model year', 'origin', 'name']
5 print(df.head(3))
6 print('\n')
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	\
0	18.0	8	307.0	130.0	3504.0	12.0	70	
1	15.0	8	350.0	165.0	3693.0	11.5	70	
2	18.0	8	318.0	150.0	3436.0	11.0	70	

	origin	name
0	1 chevrolet	chevelle malibu
1	1 buick	skylark 320
2	1 plymouth	satellite

```
In [3]: 1 mpg_to_kpl = 1.60934 / 3.78541
2
3 df['kpl'] = df['mpg'] * mpg_to_kpl
4 print(df.head(3))
5 print('\n')
6
7 df['kpl'] = df['kpl'].round(2)
8 print(df.head(3))
```

Conversión de millas por galón a kilómetros por litro

Preprocesado de datos

Ingeniería de características

Ejemplo

```
In [5]: print(df['origin'].unique())
```

```
[1 3 2]
```

```
In [6]: 1 df['origin'].replace({1:'USA', 2:'EU', 3:'JAPAN'}, inplace=True)
        2
        3 print(df['origin'].unique())
        4 print(df['origin'].dtypes)
        5 print('\n')
```

```
['USA' 'JAPAN' 'EU']
object
```

```
In [7]: 1 df['origin'] = df['origin'].astype('category')
        2 print(df['origin'].dtypes)
        3
        4 df['origin'] = df['origin'].astype('str')
        5 print(df['origin'].dtypes)
```

```
category
object
```

Conversión de tipos

Preprocesado de datos

Ingeniería de características

Ejemplo

1. Limpieza de datos

2. Conversión de tipos

3. División en 3 bins

4. Renombramos bins

5. One-hot encoding con el método `get_dummies`

```
In [12]: df = pd.read_csv('./auto-mpg.csv', header=None)

df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
              'acceleration', 'model year', 'origin', 'name']

df['horsepower'].replace('?', np.nan, inplace=True)
df.dropna(subset=['horsepower'], axis=0, inplace=True)
df['horsepower'] = df['horsepower'].astype('float')

count, bin_dividers = np.histogram(df['horsepower'], bins=3)
bin_names = ['Low output', 'Normal output', 'High output']
```

```
In [13]: df['hp_bin'] = pd.cut(x=df['horsepower'],
                              bins=bin_dividers,
                              labels=bin_names,
                              include_lowest=True)

horsepower_dummies = pd.get_dummies(df['hp_bin']).astype('int')
print(horsepower_dummies.head(15))
```

Preprocesado de datos

Ingeniería de características

Ejemplo

		Low output	Normal output	High output
Resultado	0	0	1	0
	1	0	1	0
	2	0	1	0
	3	0	1	0
	4	0	1	0
	5	0	0	1
	6	0	0	1
	7	0	0	1
	8	0	0	1
	9	0	0	1
	10	0	0	1
	11	0	1	0
	12	0	1	0
	13	0	0	1
	14	1	0	0

Conclusiones

Qué hemos aprendido

- Manejar **Series** y **DataFrames**
- **Indexar** datos en estas estructuras de datos
- Manipular los datos **operando** con ellos
- **Fusionar** y **concatenar** DataFrames
- **Agrupar** y **pivotar** los DataFrames para ordenar los datos por alguna de sus características
- **Preprocesar** datos (datos faltantes, conversión de tipos, *one-hot encoding*, eliminación de duplicados,...)